TECHNICAL CASE STUDY

Restaurant Menu Pricing *RAG*

A retrieval system over scraped restaurant menus that produces cuisine-, tier-, and location-aware pricing comparisons — with confidence reported from the real sample size.

DISCIPLINE

RAG systems · Data engineering · ETL at scale

STACK

Python · Postgres + pgvector · Claude · OpenAI · FastAPI

FORMAT

Engineering-level walkthrough of architecture, methodology & tradeoffs

CONTENTS

What's Inside

SECTION 01

Executive Summary

Restaurants set menu prices mostly by feel, with occasional competitor spot-checks. The useful question is narrow and local: for a casual-dining Mexican restaurant in this part of town, what does a carne asada taco actually go for?

Answering that well means comparing like with like (a taqueria taco is not a fine-dining taco), accounting for location (the same dish sells for more in an affluent area), and being honest about how thin the evidence is.

This project is a retrieval-augmented generation system that answers that question from public menu data. It scrapes restaurant websites, extracts structured menu items, embeds them, and stores them in Postgres with pgvector. A query embeds the dish, retrieves semantically similar items across restaurants, aggregates them deterministically by cuisine, price tier, serving type, and geographic submarket, and has a language model write the recommendation grounded in those numbers. It runs as a small HTTP service so a single dish, cuisine, and location query returns the segmented pricing as JSON.

> **THE HONEST FINDING**
>
> The modeling is the easy part. A vector search plus a grounded language-model summary is reliable and was working early. What actually gates a trustworthy pricing answer is data — in two senses. First, how many independent restaurants you have for a given cuisine, tier, and location, and how much of the public web you can even reach. Second, and more subtle, whether the data you retrieved is really the same product: a taco query that quietly pools in burritos and combo plates produces a confident, wrong number.

The engineering that earned its keep was the acquisition pipeline, the comparison hygiene that keeps like with like, and the honesty layer that refuses to overclaim. This is a standalone portfolio project, not connected to any employer or commercial system, and it stores only derived facts — dish names, prices, embeddings — never copies of menus or images.

SECTION 02

The Problem

Menu pricing intelligence has to clear several bars that simple competitor averages skip.

- **Comparing like with like.** A single average over a taqueria and a white-tablecloth restaurant is misleading, because they are different products at different positions in the market. The same trap exists within a single restaurant: a single taco, a three-taco plate with rice and beans, and a carne asada burrito are different products, and blending them produces a number that describes nothing.
- **Accounting for location.** The same cuisine at the same tier commands a real premium in an affluent submarket and a discount in a value market. A metro-wide number hides this and would tell a premium-area operator to leave money on the table and a value-area operator to price above what the market bears.
- **Being honest about confidence.** A median computed from three restaurants is a guess. A pricing tool that presents it with the same authority as one computed from thirty is worse than no tool, because it invites bad decisions with false confidence.

SECTION 03

System Architecture

The system has two halves: a fast query path that answers pricing questions, and a slower ingestion pipeline that keeps the corpus current.

Query path — retrieval-augmented generation

STEP 1 — EMBED

Dish to vector

The dish query is embedded with an OpenAI embedding model so it can be compared by meaning, not by wording.

STEP 2 — RETRIEVE

pgvector cosine search

Retrieves semantically similar menu items across restaurants. This is what lets "carne asada tacos" match "asada street taco" and "grilled steak taco" across menus that word things differently.

STEP 3 — AGGREGATE

Deterministic stats

Comparables are filtered to the queried dish form, then aggregated in SQL and Python by cuisine, tier, serving type, and zone.

STEP 4 — SYNTHESIZE

Grounded recommendation

Claude writes the recommendation grounded in those aggregated figures. The model explains the numbers; it does not invent them. Synthesis is optional, so bulk scoring can skip it.

Ingestion — the harder half

Menus appear in many formats, so the pipeline tries the cheapest path first and escalates only when it has to:

- Static HTML and schema.org JSON-LD when present.

- JavaScript-rendered single-page sites, via a headless browser (Playwright).
- Menus embedded in third-party iframe widgets (BentoBox, Toast, Square).
- Image menus, read with a vision model.
- PDF menus, read with the model's native document support.

Extraction uses tool-use, so the model returns schema-validated structured data rather than free text that has to be parsed. Long menus are chunked, and image menus are batched, so a single oversized request never silently drops items. Because scraping the open web is unreliable, the batch ingester runs each site in an isolated subprocess with a hard per-site time budget: a wedged page or a hung browser is killed at the process-group level, so one bad site can never stall a whole run.

Tech stack

Language	Python 3.12
Store	Postgres + pgvector (Supabase)
Embeddings	OpenAI text-embedding-3-small (1536-dim)
Extraction	Claude Haiku (text and image) via tool-use
Synthesis & OCR	Claude Sonnet
Rendering	Playwright (headless Chromium)
Discovery	Google Places API (New)
Interface	FastAPI service (containerized), with a Typer CLI for operations
Quality	Pure-function regression suite over pricing, tier, zone, and parsing logic

SECTION 04

Methodology

Structured extraction

Every item is extracted with four classifications that the downstream logic depends on: a serving type (à la carte, plate, combo, or side), a unit count for normalization (3 for a three-taco plate), a menu type (regular or catering), and the price kept both as the raw string and a parsed number. Keeping the raw price string matters, because real menus say "Market Price", "12/16/20", and "$8 ea", and a numeric column alone would silently drop or mangle those.

Retrieval, and keeping like with like

Comparables come from an exact pgvector nearest-neighbor search (no approximate index is needed at this corpus size). A similarity threshold drops loosely related items so a taco query does not pull in bone marrow. But similarity alone is not enough, and this was the most instructive finding of the project.

An embedding rates "carne asada burrito", "carne asada quesadilla", and "carne asada fries" as very close to "carne asada tacos", because they share the protein, even though they are different products at different prices. Pooling them inflated the figures. The fix is a **dish-form filter**: the comparable set is restricted to items that share the queried form (taco, pizza, burger, and so on), so a taco query aggregates only tacos.

> **DESIGN NOTE**
>
> This correctly shrinks the sample and lowers the reported confidence, which is the honest outcome rather than a regression. The matched set is then partitioned and summarized in code, which keeps the statistics auditable and the language model's job limited to explanation.

SECTION 05

The Pricing Model

Tiers are per cuisine, computed from the data

A single dollar boundary for "value vs upscale" is meaningless across cuisines. In the working corpus the median restaurant prices out very differently by cuisine:

$19.00	$16.50	$13.60	$13.25	$12.00
AMERICAN	ITALIAN	MEXICAN	PIZZA	BARBECUE

So $15 is "value" for American and "upscale" for barbecue. Tier boundaries are therefore derived from each cuisine's own price distribution once there are enough restaurants to support it, which also means they track inflation automatically rather than drifting out of date.

The sample size is restaurants, not items

Items within one restaurant are correlated, because a restaurant prices its whole menu high or low together. Reporting "40 priced items" when they come from four restaurants overstates confidence badly. The system counts distinct restaurants and labels every recommendation **insufficient** (under 5), **low** (5 to 9), **moderate** (10 to 19), or **high** (20 or more). The recommendation opens by stating that level and the count behind it.

The per-item price is à la carte only

A single à la carte taco and a three-taco plate with rice and beans are different products, and the second is not three of the first. An earlier version divided plate prices by their item count to recover a per-item figure, but that was wrong: a plate bundles sides, so dividing it attributes the cost of the rice and beans to the taco and overstates it.

The corrected design reports the à la carte (single-item) price as the clean per-unit comparable and leads with it. Plates are kept in their own bucket as whole-meal prices and treated as directional. Those plates are further split into bare versus with-sides, detected from the item name and description, so a plain plate is never averaged against one that includes rice and beans. Every figure also reports the share of the comparable set it rests on, so a thin per-item read announces itself.

Catering is held out

Catering menus are priced per tray or per person and would corrupt both dine-in price statistics and the tier calibration. They are tagged at extraction and excluded from regular pricing and from the tier boundaries.

> **A BUG THAT SURFACED HERE**
>
> An early run let a catering-only PDF inflate a restaurant's median and mislabel its tier — exactly the kind of contamination the separation is meant to prevent. Fixed by tagging at extraction time so the filter applies everywhere downstream, not just at the query.

SECTION 06

Geographic Submarkets

The metro is divided into four coarse zones — a premium northeast, a central core, an east valley, and a west valley — assigned from each restaurant's ZIP. Pricing reports the query's zone figure next to the metro figure, each with its own confidence. The zone figure is computed on the same tier-filtered comparable set, so it isolates the location effect rather than picking up a zone's tier mix.

An earlier design that used a single zone-vs-metro multiplier was rejected during review for exactly that confound: it would have conflated a genuine location premium with the fact that one zone happens to hold pricier restaurants.

THE LESSON GENERALIZES

Getting the geography right depended on a parsing detail that was quietly wrong at first. The address parser grabbed the first five-digit run in each address, which is the street number when it happens to be five digits, not the postal code, and that silently misassigned about a quarter of the corpus to the wrong submarket. The parser now anchors on the two-letter state code, and the already-stored rows were repaired by re-fetching each restaurant's address and matching it back by street number, bringing the corpus to 494 of 500 correctly located. A model is only as good as the join keys underneath it, and a silent parsing bug is more dangerous than a loud one.

SECTION 07

Privacy by Construction

Output never names an individual restaurant. Identity is stripped from the data before it reaches the language model, so a specific restaurant cannot be named even by accident, rather than relying on a prompt instruction the model might ignore. The model is also instructed never to identify a place indirectly, by location or description.

Output speaks only in category terms — for example, "*casual-dining Mexican restaurants in the premium zone charge around X for carne asada tacos.*" This mirrors how this kind of competitive data is handled responsibly: aggregate, never attributed.

SECTION 08

Data Collection & Corpus

Restaurants are discovered through the Google Places API by cuisine and city, which returns names and websites. A resumable batch job runs each website through the ingestion pipeline, deduplicating on URL and name, logging every failure with its cause, and recovering bot-blocked sites by retrying through the headless browser.

The working corpus spans five cuisines across the metro, measured in **priced restaurants** — the real sample size:

71	63	53	51	40
MEXICAN	AMERICAN	PIZZA	ITALIAN	BARBECUE

At the metro level all five are in high-confidence territory. By submarket, the premium, central, and east-valley zones reach moderate-to-high confidence for the major cuisines, which makes cross-zone comparisons defensible for those. Targeted collection lifted the suburban west valley to moderate for most cuisines as well.

Barbecue is the exception everywhere: there are simply few barbecue restaurants with scrapeable sites, so its zone-level samples stay thin no matter how hard you pull — which the confidence labels report honestly rather than paper over.

SECTION 09

Limitations & Next Steps

This project taught me more about the messiness of public data than about modeling, and the limitations are worth stating plainly.

A meaningful share of restaurant sites block automated requests, and some block headless browsers too. A real browser recovers many of them automatically — it passes the bot checks that a plain HTTP client fails — but the hardest sites need either manual capture or infrastructure I chose not to build for a portfolio project. Roughly a third of attempts are lost to dead links, blocks, or off-site menus, which is normal for scraping the open web. Many restaurants no longer host their own menu at all; it lives on a delivery platform or social media, out of reach of any scraper.

Coverage is gated by supply, not just effort. Some cuisine-and-zone cells simply do not have twenty independent restaurants with scrapeable menus in the data, and barbecue is the clearest case. The right response is to report low confidence, not to invent precision.

Extraction quality is the other ceiling. Serving type and unit count are inferred by a model and are occasionally wrong, which is why the per-item figure leans on à la carte items and the plate figures are treated as directional. Vision and PDF extraction are accurate but the slow and expensive path, so the pipeline avoids them unless the cheaper paths fail.

> **REGRESSION COVERAGE**
>
> The pricing, tier, zone, and parsing rules are now covered by a fast, pure-function regression suite, so the corrections described above — à la carte-only per-item, the dish-form filter, the ZIP parser — cannot silently regress in a later edit.

SECTION 10

Honest Framing

The interesting result of this project is not a model score. It is the demonstration that a useful pricing answer is mostly a data-acquisition and data-honesty problem, and that the right engineering response is to invest in reaching the data, comparing like with like, segmenting it correctly, and refusing to overclaim when it is thin.

THE THESIS IN ONE LINE

The system is built to say "not enough data yet" clearly and often, and to shrink its own confidence when a stricter, more correct comparison leaves a smaller sample — because in this domain that honesty is the most valuable thing it can tell an operator.

— END —